

MEMORIA TÉCNICA

Proyecto g_agenda

Sistema de generación automatizada de agendas de ocupación de espacios

Grupo de desarrollo: ROSIJO

Integrantes:

❖ **José Antonio Taborda Morán**

Módulo de Persistencia, Archivos Físicos y Tiempo

❖ **Roberth Altamar**

Módulo de Lógica de Peticiones y Control de Conflictos

❖ **Simon Saavedra Alfaro**

Módulo Frontend, Estructuras Visuales e Integración

Programación · IFCD0210

Desarrollo de Aplicaciones con Tecnologías Web

22 de julio de 2026

Índice

1. Introducción y objetivos
2. Análisis de requisitos
3. Arquitectura de la solución
4. Estructura del proyecto
5. Descripción de las clases
6. Formatos de entrada
7. Formatos de salida
8. Algoritmos principales
9. Decisiones de diseño
10. Internacionalización
11. Tratamiento de errores y validaciones
12. Instrucciones de compilación y ejecución
13. Gestión de incidencias, pruebas y depuración
14. Reparto del trabajo
15. Conclusiones

1. Introducción y Objetivos

El presente documento recoge la memoria técnica del proyecto g_agenda, desarrollado como trabajo final del módulo de Programación. La aplicación tiene como finalidad generar de forma automatizada la agenda mensual de ocupación de un conjunto de espacios (salas de conferencias, aulas o instalaciones similares) a partir de un fichero de peticiones de reserva.

El programa parte exclusivamente de ficheros de texto como fuente de datos y produce como resultado un conjunto de ficheros HTML —uno por cada espacio gestionado— junto con un fichero de registro que documenta las incidencias detectadas durante el proceso de asignación. En ningún momento se solicita información por pantalla ni se muestran resultados por consola, cumpliendo así el requisito de que toda la entrada y salida se realice mediante ficheros.

Objetivos técnicos

- Automatizar la conversión de un fichero de peticiones en calendarios web sin intervención manual.
- Resolver de forma determinista los conflictos de ocupación entre actividades mediante reglas de prioridad.
- Calcular con exactitud la estructura del calendario (número de días, día de la semana y semana del mes) empleando la biblioteca nativa java.time.
- Permitir que la aplicación opere en distintos idiomas sin modificar el código fuente, mediante ficheros externos de internacionalización.
- Construir una arquitectura modular que permita el desarrollo paralelo y el mantenimiento independiente de cada componente.

2. Análisis de requisitos

A partir del enunciado se identificaron los siguientes requisitos funcionales, que han condicionado el diseño de la aplicación:

Requisito	Descripción
Entrada por ficheros	El programa lee la configuración general, las peticiones de reserva y los diccionarios de idioma desde ficheros de texto ubicados en la raíz del proyecto.
Salida por ficheros	Genera un fichero HTML por cada espacio y un fichero de incidencias. No se produce salida por pantalla.
Períodos y máscaras	Cada petición define un rango de fechas, una máscara de días de la semana y una máscara de franjas horarias, que deben desglosarse en asignaciones concretas.
Prioridad de cierre	La actividad de cierre debe procesarse en primer lugar, con independencia de su posición en el fichero de peticiones, y se representa con sombreado gris.
Acumulación por actividad	Las peticiones de una misma actividad se agrupan y conservan la prioridad de su primera aparición, aunque aparezcan separadas en el fichero.
Registro de conflictos	Las asignaciones que no pueden realizarse por solapamiento se documentan en el fichero de incidencias.
Multiidioma	El idioma de los ficheros de entrada y de los ficheros de salida es configurable y se resuelve mediante ficheros de internacionalización.

Requisito	Descripción
Calendario variable	El número de semanas mostradas debe calcularse dinámicamente, ya que un mes puede ocupar entre cuatro y seis filas.

3. Arquitectura de la solución

La aplicación se ha estructurado siguiendo una arquitectura desacoplada en **tres etapas secuenciales**, en la que cada módulo tiene una responsabilidad única y se comunica con el siguiente mediante estructuras de datos bien definidas. Este planteamiento permite que un módulo pueda modificarse o sustituirse sin afectar a los demás, siempre que respete el contrato de entrada y salida acordado.

Etapas 1 — Configuración y tiempo	Etapas 2 — Lógica y reservas	Etapas 3 — Generación web
Lee la configuración general, carga los diccionarios de idioma en memoria y proporciona las funciones de cálculo de calendario al resto de módulos.	Interpreta las peticiones, valida su coherencia, aplica las reglas de prioridad, rellena la matriz de ocupación y registra los conflictos.	Recorre las matrices resultantes, aplica las traducciones del idioma de salida y construye los ficheros HTML de cada espacio.

El flujo completo de ejecución es el siguiente:

```

config.txt + peticions.txt + internacional.ESP / internacional.ENG
    ↓
Etapa 1: configuración e idiomas cargados en memoria
    ↓
Etapa 2: matriz de ocupación por espacio + incidencias
    ↓
Etapa 3: Sala1.html · Sala2.html · incidencias.log

```

4. Estructura del proyecto

Los ficheros fuente se agrupan en el directorio src, mientras que los ficheros de datos y los resultados generados se ubican en la raíz del proyecto. Esta separación es necesaria porque la aplicación resuelve las rutas de los ficheros de datos respecto al directorio de ejecución.

```

Proyecto_Final_Agenda/
├── src/
│   ├── Configuracion.java      Modelo de parámetros generales
│   ├── GestionDatosTiempo.java Lectura de configuración, idiomas y calendario
│   ├── Peticion.java           Modelo de una petición de reserva
│   ├── ProcesadorAgenda.java   Motor de procesamiento y conflictos
│   ├── GeneradorHTML.java      Construcción de los ficheros HTML
│   └── Main.java               Coordinación del flujo completo
├── config.txt                  Año, mes e idiomas
├── peticions.txt               Peticiones de reserva
├── internacional.ESP           Diccionario de entrada
└── internacional.ENG          Diccionario de salida

```

5. Descripción de las clases

5.1. Configuracion

Clase de modelo que encapsula los cuatro parámetros generales de ejecución: año, mes, idioma de entrada e idioma de salida. Expone únicamente métodos de acceso, de forma que el resto de la aplicación consulta la configuración sin poder alterarla una vez construida.

5.2. GestionDatosTiempo

Módulo de persistencia y cálculo temporal. Agrupa tres responsabilidades relacionadas con la preparación del entorno de ejecución:

- **Lectura de la configuración:** interpreta config.txt y construye el objeto Configuracion correspondiente.
- **Gestión de diccionarios:** carga los ficheros de internacionalización en una estructura HashMap que asocia cada código numérico con su literal, permitiendo la consulta directa de cualquier término.
- **Cálculo de calendario:** mediante java.time.LocalDate determina el número de días del mes, el día de la semana correspondiente a una fecha y la fila del calendario en la que debe representarse cada día.

Adicionalmente, proporciona la traducción entre las letras de la máscara de días y su valor numérico, lo que permite al módulo de lógica interpretar las máscaras con independencia del idioma configurado.

5.3. Peticion

Clase de modelo que representa una línea del fichero de peticiones. Almacena la actividad, el espacio, las fechas de inicio y fin —convertidas a objetos LocalDate— y las máscaras de días y horas. Su uso permite trabajar con objetos tipados en lugar de con cadenas de texto durante todo el procesamiento.

5.4. ProcesadorAgenda

Constituye el núcleo lógico de la aplicación. Se encarga de la lectura del fichero de peticiones, de su validación, de la ordenación por prioridad, de la construcción y relleno de las matrices de ocupación y del registro de las incidencias. Mantiene además los contadores de horas asignadas y no asignadas por actividad.

5.5. GeneradorHTML

Responsable de la presentación. Recibe la matriz de ocupación de un espacio, la configuración y el gestor de idiomas, y construye el documento HTML completo en memoria mediante StringBuilder antes de volcarlo a disco en una única operación de escritura. Divide la agenda en tablas semanales independientes y aplica el sombreado correspondiente a las franjas cerradas.

5.6. Main

Clase coordinadora. Establece el orden de ejecución de los módulos y actúa como único punto de entrada de la aplicación, sin contener lógica de negocio propia.

6. Formatos de entrada

6.1. Fichero de configuración

El fichero `config.txt` contiene dos líneas. La primera indica el año y el mes que se desea procesar; la segunda, el idioma en que están redactadas las peticiones y el idioma en que debe generarse la salida.

```
2025 11
esp ENG
```

En el ejemplo, la aplicación procesa noviembre de 2025, interpreta las peticiones en español y genera los ficheros HTML en inglés.

6.2. Fichero de peticiones

Cada línea del fichero `petitions.txt` describe una petición de reserva mediante seis campos separados por espacios:

```
Cerrado Sala1 01/01/2025 31/12/2025 LMXJVSD 00-07_21-24
|         |         |         |         |         |
actividad espacio f.inicial f.final m. días m. horas
```

Campo	Significado
Actividad	Nombre de la actividad, sin espacios internos. El valor reservado de cierre recibe tratamiento prioritario.
Espacio	Nombre del espacio o sala al que se aplica la reserva.
Fecha inicial	Inicio del período de reserva, en formato dd/mm/aaaa.
Fecha final	Fin del período de reserva, en formato dd/mm/aaaa.
Máscara de días	Conjunto de iniciales de los días de la semana en los que debe aplicarse la reserva.
Máscara de horas	Conjunto de franjas horarias separadas por guión bajo. Cada franja indica hora de inicio y de fin.

La máscara de horas admite hasta cinco franjas. Una expresión como `08-10_19-21` se interpreta como la reserva de las horas 8 y 9 por un lado, y 19 y 20 por otro, siempre dentro del mismo día.

6.3. Ficheros de internacionalización

Los ficheros `internacional.ESP` e `internacional.ENG` asocian cada código numérico con su literal correspondiente, separados por punto y coma. Su estructura permite añadir nuevos idiomas creando únicamente un fichero adicional, sin modificar el código fuente.

```
001;Agenda
002;Lunes,Martes,Miércoles,Jueves,Viernes,Sábado,Domingo
003;LMXJVSD
007;Cerrado
```

7. Formatos de salida

7.1. Ficheros HTML de ocupación

Por cada espacio presente en el fichero de peticiones se genera un documento HTML independiente, denominado con el nombre del espacio. El documento incluye una cabecera con el título de la agenda, el nombre del espacio y el mes y año procesados, seguida de una tabla por cada semana del mes.

Cada tabla presenta las veinticuatro franjas horarias en filas y los siete días de la semana en columnas. Las celdas correspondientes a la actividad de cierre se representan con fondo gris; las celdas ocupadas por una actividad muestran su denominación sobre fondo diferenciado; y las celdas libres o ajenas al mes se dejan en blanco.

7.2. Fichero de incidencias

El fichero incidencias.log recoge todas las asignaciones que no han podido realizarse por encontrarse la franja previamente ocupada. Cada registro identifica la actividad afectada, el espacio, el día y la hora del conflicto, así como la actividad que ocupaba la posición.

```
Conflicto: ReunioPerl no pudo entrar en Sala1 dia 3 hora 14 (ocupado por ReunioJava)
```

8. Algoritmos principales

8.1. Cálculo de la posición en el calendario

La representación de un mes en forma de cuadrícula requiere determinar en qué fila y en qué columna debe situarse cada día. La columna se obtiene directamente del día de la semana. Para la fila se calcula el desfase inicial del mes —esto es, el número de posiciones vacías que preceden al día uno— y se suma al día en curso, dividiendo el resultado entre siete.

```
desfase = díaDeLaSemana(primer día del mes) - 1  
fila = (día - 1 + desfase) / 7
```

Este planteamiento resuelve automáticamente el número variable de semanas: un mes que comienza en sábado ocupa cinco filas, mientras que otras combinaciones pueden requerir cuatro o seis, sin necesidad de fijar ningún valor en el código.

8.2. Desglose de las máscaras horarias

La interpretación de la máscara de horas se realiza en dos niveles. En primer lugar se separa la cadena por el carácter de subrayado, obteniendo las franjas independientes; a continuación, cada franja se divide por el guión para extraer la hora inicial y la final. El intervalo resultante se recorre generando las horas concretas que deben ocuparse, excluyendo la hora final para evitar solapamientos entre franjas consecutivas.

8.3. Filtrado de días aplicables

Para cada petición se recorren todos los días del mes procesado y se seleccionan aquellos que cumplen simultáneamente dos condiciones: encontrarse dentro del período comprendido entre la fecha inicial y la final, y coincidir con alguno de los días indicados en la máscara. La correspondencia entre la letra de la máscara y el día de la semana se resuelve consultando el diccionario del idioma de entrada.

8.4. Ordenación por prioridad y resolución de conflictos

Antes de rellenar las matrices, la lista de peticiones se reordena situando en primer lugar las correspondientes a la actividad de cierre, y agrupando a continuación el resto por nombre de actividad, respetando el orden de primera aparición. De este modo se garantiza que los cierres queden asignados antes que cualquier reunión y que las peticiones de una misma actividad conserven su prioridad aunque aparecen dispersas en el fichero.

Durante el relleno, cada posición de la matriz se comprueba antes de escribir. Si la celda está libre, se asigna la actividad y se incrementa el contador de horas asignadas. Si ya se encuentra ocupada, no se sobrescribe: se registra la incidencia correspondiente y se incrementa el contador de horas no asignadas de la actividad rechazada.

9. Decisiones de diseño

9.1. Matriz de ocupación

Se ha optado por representar la ocupación de cada espacio mediante una matriz bidimensional en la que la primera dimensión corresponde a las horas del día y la segunda a los días del mes. Esta estructura ofrece acceso directo a cualquier posición sin necesidad de recorrer colecciones, y simplifica notablemente la comprobación de conflictos, que se reduce a verificar si una posición concreta está ocupada.

9.2. Diccionarios en memoria

Los ficheros de internacionalización se cargan en una estructura de tipo HashMap. La elección responde a que el acceso se realiza siempre por clave conocida —el código del término—, situación en la que esta estructura resulta más eficiente que una lista, al no requerir recorrido secuencial.

9.3. Construcción del HTML en memoria

El documento HTML se construye íntegramente mediante StringBuilder y se escribe en disco en una única operación. Esta decisión reduce el número de accesos al sistema de ficheros y evita la penalización asociada a la concatenación repetida de cadenas inmutables.

9.4. Separación de idioma de entrada y de salida

La aplicación mantiene dos diccionarios diferenciados durante la ejecución. El diccionario de entrada se emplea para interpretar las máscaras y los términos del fichero de peticiones, mientras que el de salida se utiliza exclusivamente para generar los literales de los documentos HTML. Esta separación permite leer peticiones en un idioma y publicar la agenda en otro.

10. Internacionalización

La totalidad de los literales que aparecen en los ficheros generados procede de los ficheros de internacionalización. El código fuente no contiene ningún texto visible para el usuario final, de modo que la modificación del idioma de salida se realiza editando una única línea del fichero de configuración.

Código	Español	Inglés
001	Agenda	Schedule
002	Lunes, Martes, Miércoles...	Monday, Tuesday, Wednesday...
003	LMXJVSD	MTWHFSN
004	Enero, Febrero, Marzo...	January, February, March...
005	Año, Mes, Semana, Día	Year, Month, Week, Day
006	Generado por	Generated by
007	Cerrado	Closed
008	Error	Error

El **código 003** merece una mención específica, ya que define las iniciales empleadas en las máscaras de días. Al obtenerse del diccionario y no del código fuente, la aplicación puede interpretar máscaras redactadas en cualquier idioma para el que exista fichero de internacionalización.

11. Tratamiento de errores y validaciones

La aplicación se ha diseñado bajo el principio de desconfiar de los datos de entrada. Se han implementado las siguientes comprobaciones:

- **Ficheros inexistentes o vacíos:** la lectura se realiza mediante **try-with-resources**, de modo que los recursos se liberan aun cuando se produzca un error. Si la configuración no puede obtenerse, la ejecución finaliza de forma controlada.
- **Líneas vacías o comentarios:** durante la carga de los diccionarios se ignoran las líneas en blanco y aquellas que comienzan por almohadilla, evitando que se registren entradas inválidas.
- **Coherencia de fechas:** se descartan las peticiones cuya fecha final es anterior a la inicial, situación que impide determinar un período válido.
- **Validez de las franjas horarias:** se rechazan las franjas cuya hora final no es posterior a la inicial, así como aquellas que implicaría el cruce de la medianoche.
- **Celdas sin asignación:** durante la generación del HTML se comprueba que la posición consultada contenga información antes de escribirla, evitando la aparición de valores nulos en el documento final.

La continuidad de la ejecución de las peticiones descartadas no interrumpen la ejecución: el programa continúa procesando el resto del fichero, garantizando que un dato erróneo aislado no impida la generación de la agenda.

12. Instrucciones de compilación y ejecución

La aplicación no requiere bibliotecas externas: utiliza exclusivamente la biblioteca estándar de Java. Para su compilación y ejecución debe situarse el directorio de trabajo en la raíz del proyecto, donde residen los ficheros de datos.

Compilación:

```
javac -encoding UTF-8 -d bin src/*.java
```

Ejecución:

```
java -cp bin Main
```

El parámetro de codificación resulta necesario para el tratamiento correcto de los caracteres acentuados presentes en los ficheros de internacionalización. Una vez finalizada la ejecución, los ficheros de salida quedan disponibles en la raíz del proyecto.

13. Gestión de incidencias, pruebas y depuración

Durante la integración, el grupo sometió el software a pruebas y corrigió tres fallos semánticos detectados en la terminal de Visual Studio Code:

- **ArrayIndexOutOfBoundsException:** el generador visual se colgaba al parsear la línea del diccionario. El código invocaba el índice [4] de una colección con sólo cuatro elementos (0–3). Se sincronizaron las llamadas con las posiciones reales y se recuperó la estabilidad.
- **Solapamiento léxico (“Tancat” vs “Cerrado”):** las celdas bloqueadas no cambiaban de color porque el frontend buscaba “Tancat” y la matriz inyectaba “Cerrado”. Se unificó el condicional a **equalsIgnoreCase(“Cerrado”)**, admitiendo además las variantes “Tancat” y “Closed” para garantizar la compatibilidad independientemente del idioma de los datos de entrada.
- **Corrupción de caracteres (encoding):** los acentos y las eñes se mostraban corruptos en distintas terminales. José Antonio forzó la lectura en UTF-8 en sus flujos y Simon añadió en la cabecera del HTML.

14. Reparto del trabajo

El desarrollo se ha organizado asignando a cada integrante un módulo completo, con responsabilidad sobre su diseño, implementación y verificación. La coordinación entre módulos se ha basado en acordar previamente las estructuras de datos e interfaces compartidas.

Integrante	Módulo	Clases desarrolladas
José Antonio Taborda	Configuración, idiomas y cálculo temporal	Configuracion, GestionDatosTiempo

Integrante	Módulo	Clases desarrolladas
Robert	Lógica de reservas y control de conflictos	Peticion, ProcesadorAgenda
Simon Saavedra	Generación de documentos HTML	GeneradorHTML
Trabajo conjunto	Integración y verificación final	Main

La gestión de versiones se ha realizado mediante un repositorio compartido, lo que ha permitido el desarrollo simultáneo de los tres módulos y la consolidación posterior del producto final.

15. Conclusiones

El proyecto ha permitido construir una aplicación completa que transforma un conjunto de peticiones expresadas en texto plano en una agenda web estructurada, resolviendo de forma autónoma los conflictos de ocupación y adaptando su idioma sin intervención sobre el código.

Desde el punto de vista técnico, se han consolidado conocimientos relativos al tratamiento de ficheros, al uso de estructuras de datos apropiadas para cada necesidad, al manejo de fechas mediante la biblioteca estándar y a la programación orientada a objetos aplicada a un problema con requisitos reales.

En el plano metodológico, la experiencia ha puesto de manifiesto que, en un desarrollo repartido entre varias personas, la definición previa de los contratos entre módulos —nombres, tipos y estructuras compartidas— resulta tan determinante como la corrección de la lógica interna de cada componente. La arquitectura modular adoptada ha facilitado el aislamiento de los errores detectados y ha permitido su corrección sin afectar al resto del sistema.

El resultado es una aplicación funcional, documentada y ampliable, en la que la incorporación de nuevos espacios o de nuevos idiomas no requiere modificación alguna del código fuente.